



Lecture[08]

STACK, QUEUE



Agenda

- Stack
 - Principal ADT
 - push, pop
 - Array-based Stack
 - Linked Stack (链式栈)
- Queue
 - Principal ADT
 - enqueue, dequeue
 - Linked queue (链式队列)
 - Array-based queue/Circular list



Stacks

入栈和出栈

元素C

元素B

元素A



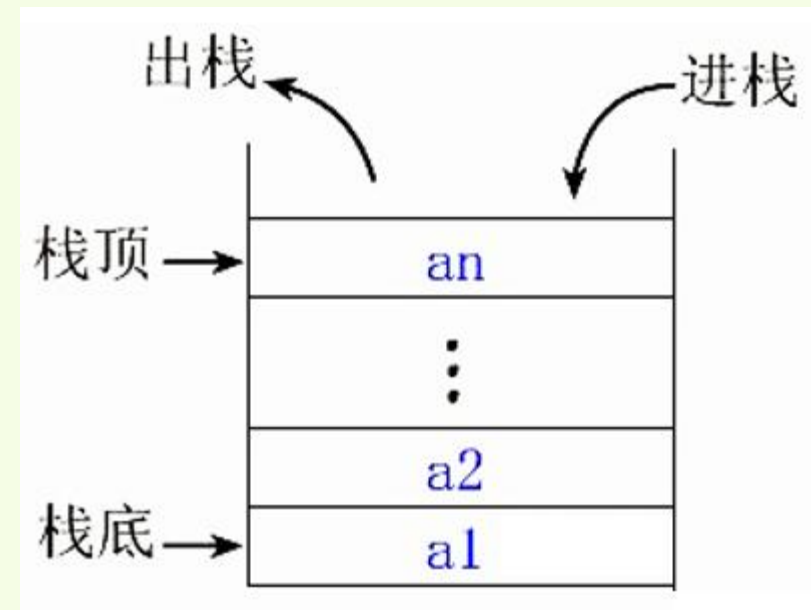
Stacks

LIFO: Last In, First Out.

Restricted form of list: Insert and remove only at front of list.

Notation:

- Insert: PUSH (进栈、入栈或压栈)
- Remove: POP (出栈或退栈)
- The accessible element is called TOP (栈顶元素) .





Stack ADT

```
typedef int BOOL ;
typedef int Elem;
    // Initialize the stack
void init(Stack);
    // Reinitialize the stack
void clear(Stack);
    // Push an element onto the top of the stack.
BOOL push(Stack, Elem);
    // Remove the element at the top of the stack.
BOOL pop(Stack, Elem*);
    // Get a copy of the top element in the stack
BOOL topValue(Stack, Elem*);
    // Return the number of elements in the stack.
int length(Stack);
```



栈-实战分析

D C B A

C,B,D,A

栈



栈顶



Array-Based Stack

```
// Array-based stack implementation
int size;           // Maximum size of stack
int top;            // Index for top element
Elem *listArray;    // Array holding elements
```

Issues:

- Which end is the top?
 - 应该是数组的最后一个元素。
- Where does “top” point to?
 - 数组的最后一个元素
- What is the cost of the operations?
 - 花费常数时间，因为只能对栈顶元素操作。



Linked Stack

```
// Linked stack implementation  
node_ptr top; // Pointer to first elem  
int currentSize; // Count number of elems
```

What is the cost of the operations?

- 花费常数时间，因为只能对栈顶元素操作。
- 链式栈主要的缺点在于调用malloc和free函数要花费时间。

How do space requirements compare to the array-based stack implementation?

- 基于数组的栈需要提前声明一个数组的大小，可能会造成一定的存储空间浪费。
- 链式栈在空间上可以更节约内存。



Array-Based Stack Class (1)

```
#define TRUE 1
#define FALSE 0
typedef int BOOL ;

typedef int Elem;

typedef struct STACK{
    int size ;      // Maximum size of stack
    int top ;       // Index for top element
    Elem *listArray ; // Array holding elements
}*Stack;
```



Array-Based Stack Class (2)

```
void init_size(Stack stack, int mysize)    //初始化栈的大小
{
    stack->size = mysize;
    stack->top = 0;
    stack->listArray = (Elem*)malloc((mysize)*sizeof(Elem));
}
```

```
void init(Stack stack)    //初始化为包含200个元素的栈
{
    init_size(stack, 200);
}
```

```
void clear(Stack stack)    //清空栈
{
    stack->size = 0;
    stack->top = 0;
    if( stack->listArray != NULL ){
        free(stack->listArray);
        stack->listArray = NULL;
    }
}
```



Array-Based Stack Class (3)

注：栈顶元素的索引是从1开始，而数组的索引是从0开始。

```
BOOL push(Stack stack, Elem e){    //入栈
    if ( stack->top < stack->size ){
        stack->listArray[ stack->top++ ] = e;
        return TRUE;
    }
    return FALSE;
}

BOOL pop(Stack stack, Elem* e){    //出栈
    if ( stack->top > 0 ){
        *e = stack->listArray[ stack->top - 1 ];
        stack->top --;
        return TRUE;
    }
    return FALSE;
}
```



Array-Based Stack Class (4)

```
// Get a copy of the top element in the stack
// 读取栈顶元素的值
BOOL topValue(Stack stack, Elem* e){
    if ( stack->top > 0 ){
        *e = stack->listArray[ stack->top - 1 ];
        return TRUE;
    }
    return FALSE;
}

// Return the number of elements in the stack.
// 返回栈中元素个数
int length(Stack stack){
    return stack->top;
}
```



Queues

入队和出队

元素C

元素B

元素A



Queues

FIFO: First in, First Out

Restricted form of list: Insert at one end, remove from the other.

Notation:

- Insert: Enqueue
- Delete: Dequeue
- First element: Front
- Last element: Rear

Classification:

- Linked queue
- Sequential queue



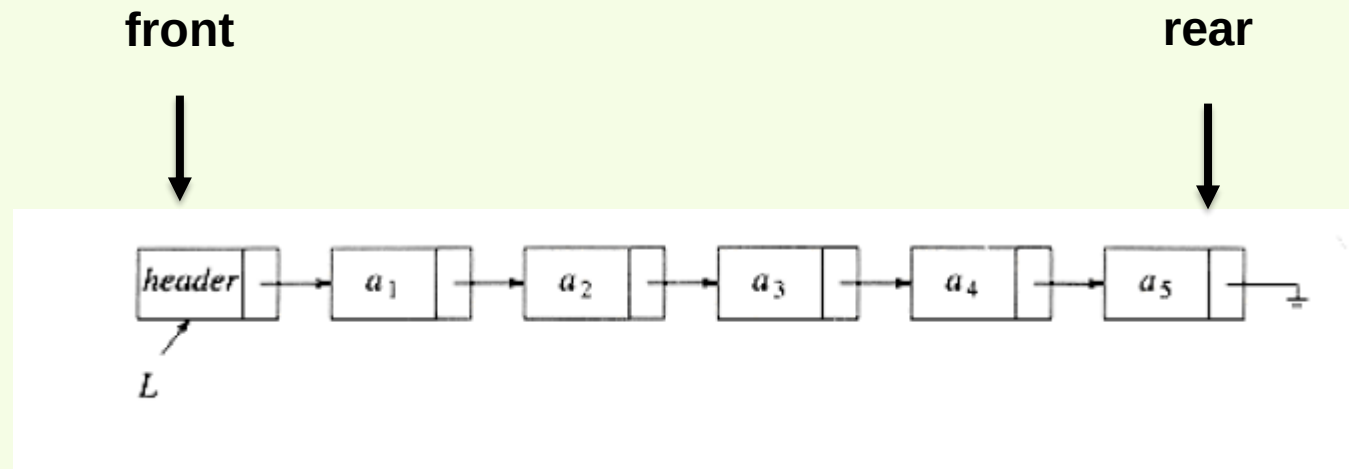
Queue ADT

```
// Initialize the queue  
void init(Queue, int);  
// Reinitialize the queue  
void clear(Queue);  
    // enqueue an element into the queue.  
BOOL enqueue(Queue, Elem) ;  
    // dequeue an element from the queue.  
BOOL dequeue(Queue, Elem*);  
    // Get a copy of the front element in the queue  
bool frontValue(Queue, Elem*);  
    // Return the number of elements in the queue.  
int length(Queue);
```



Linked Queue

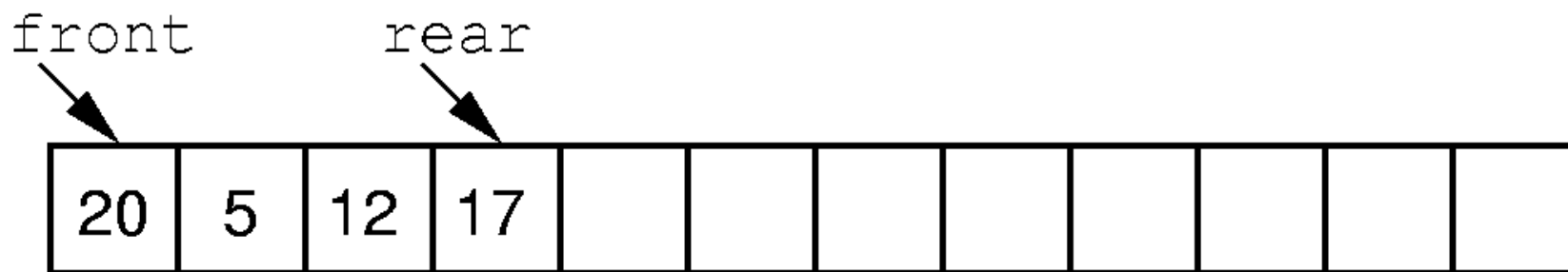
- Enqueue $\Leftrightarrow$ Append
- Dequeue $\Leftrightarrow$ remove from the head
- No need fence concept



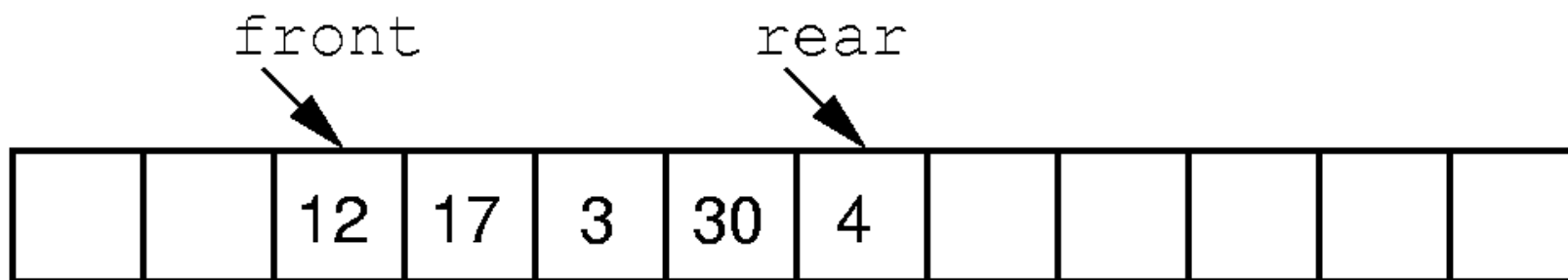
Two pointers



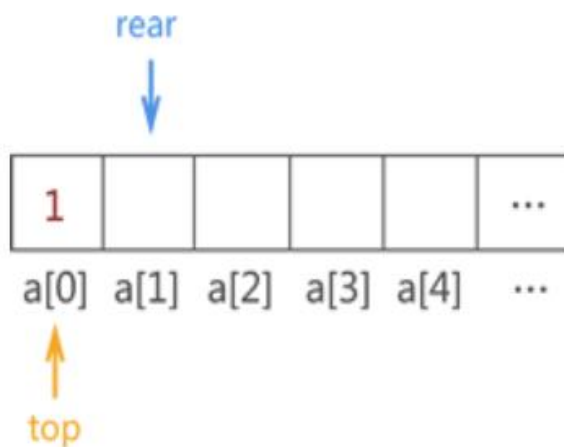
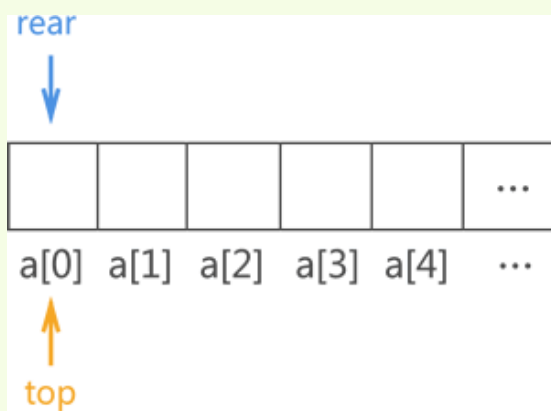
Sequential Queue Implementation (1)



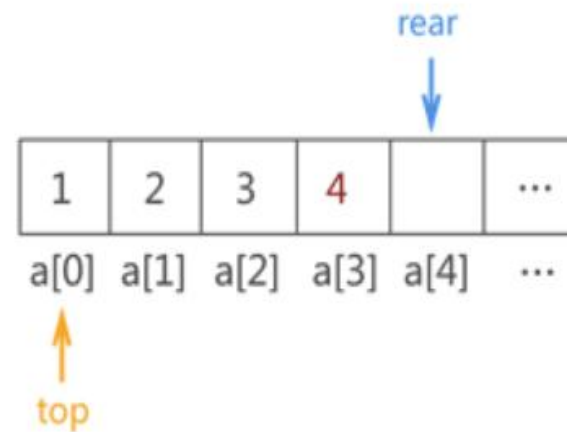
(a)



(b)



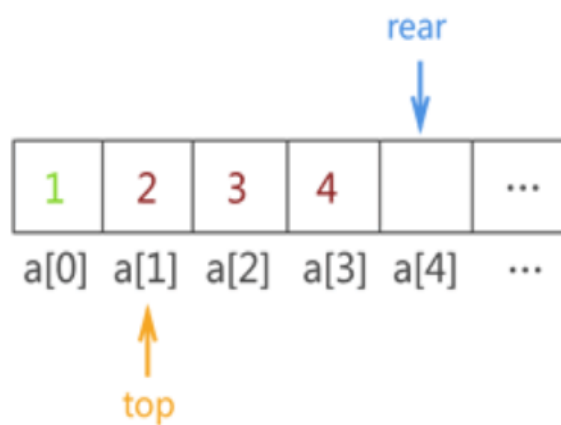
a) 元素 1 进队



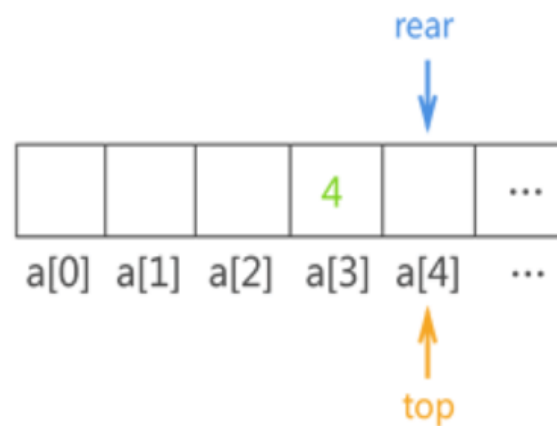
b) 元素 4 进队

图 2 数据进顺序队列的过程实现示意图

在图 2 基础上，顺序队列中数据出队列的实现过程如图 3 所示：



a) 元素 1 出队



b) 元素 4 出队

图 3 数据出顺序队列的过程示意图

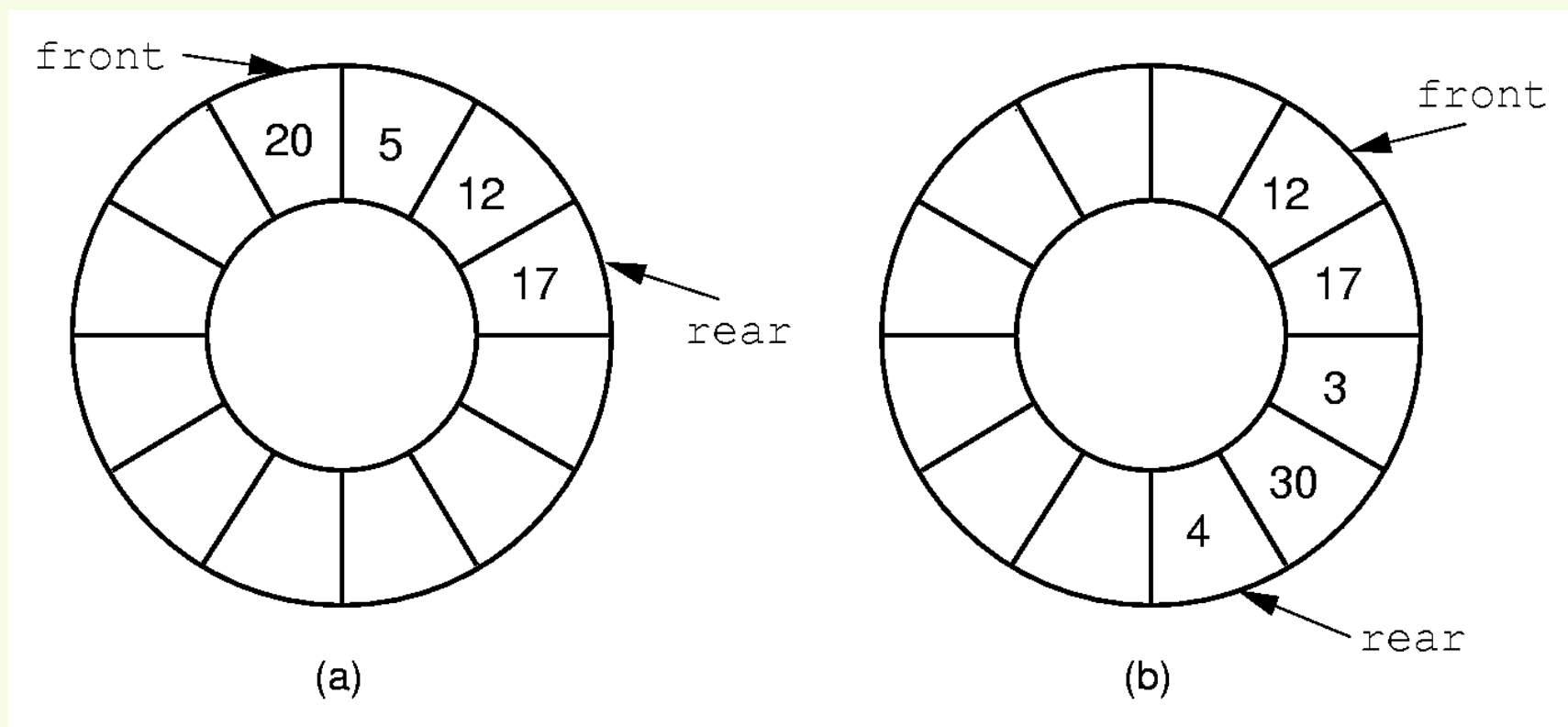


- 顺序队列的底层使用的是数组，因此需预先申请一块足够大的内存空间初始化顺序队列。
- 以数组作为底层数据结构时，队列在顺序存储上的不足：每次从数组头部删除元素（出队）后，需要将头部以后的所有元素往前移动一个位置，这是一个时间复杂度为 $O(n)$ 的操作。
- 如果说出队入队都不移动其他元素，那么我们会发现，随着不断地入队出队，队列会慢慢移到数组末尾。最终队列右边将没有存储空间，尽管这个数组里还有空间。
- 顺序队列整体后移造成的影响是：
 - 顺序队列之前的数组存储空间将无法再被使用，造成了空间浪费；
 - 如果顺序表申请的空间不够大，则直接造成程序中数组溢出，产生溢出错误；
- 我们希望队列的插入与删除操作都是 $O(1)$ 的时间复杂度，同时不会造成数组空间的浪费，我们应该使用循环队列。以数组作为底层数据结构时，一般讲队列实现为循环队列。



Sequential Queue Implementation (2)

- 可以把数组看出一个首尾相连的圆环，删除元素时将队首标志往后移动，添加元素时若数组尾部已经没有空间，则考虑数组头部的空间是否空闲，如果是，则在数组头部进行插入。





- 如何区分一个满的队列和一个空的队列？

- 为了解决这个难题，必须在队列中留下一个空槽，或者使用和外部变量来确定队列是否为空。

- 循环队列如何进行判空和满操作：

- 方法1：牺牲掉数组中的一个存储空间，尾指针的下一个位置和头指针相遇，就说明数组满了。

- 方法2：设置一个标记flag

- 初始值 $\text{flag} = 0$ ；入队列： $\text{flag} = 1$ ；出队列： $\text{flag} = 0$ ；

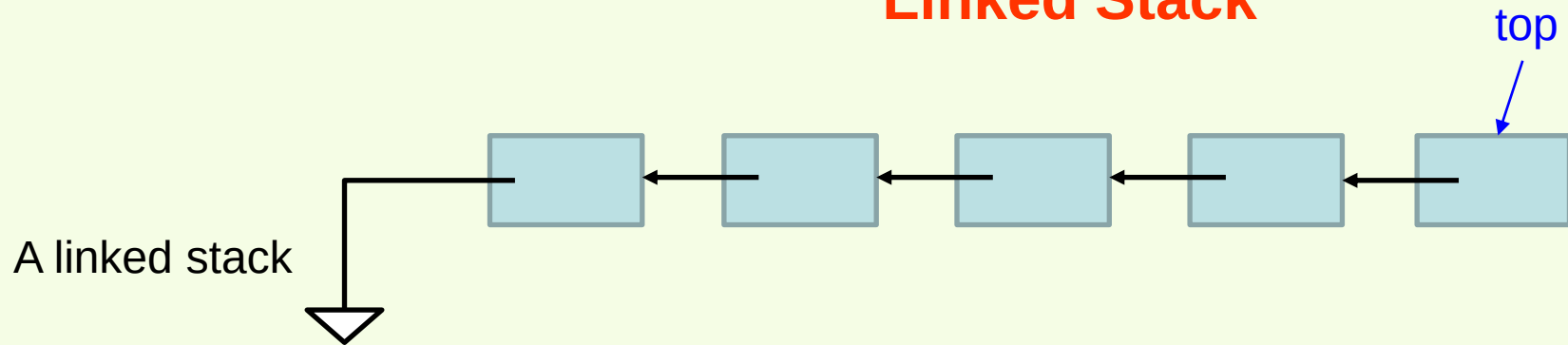
- 队列为空时：($\text{front} == \text{rear} \ \&\& \ \text{flag} == 0$) 队列的头指针等于队列的尾指针

- 队列为满时：($\text{front} == \text{rear} \ \&\& \ \text{flag} == 1$) 队列的头指针等于队列的尾指针

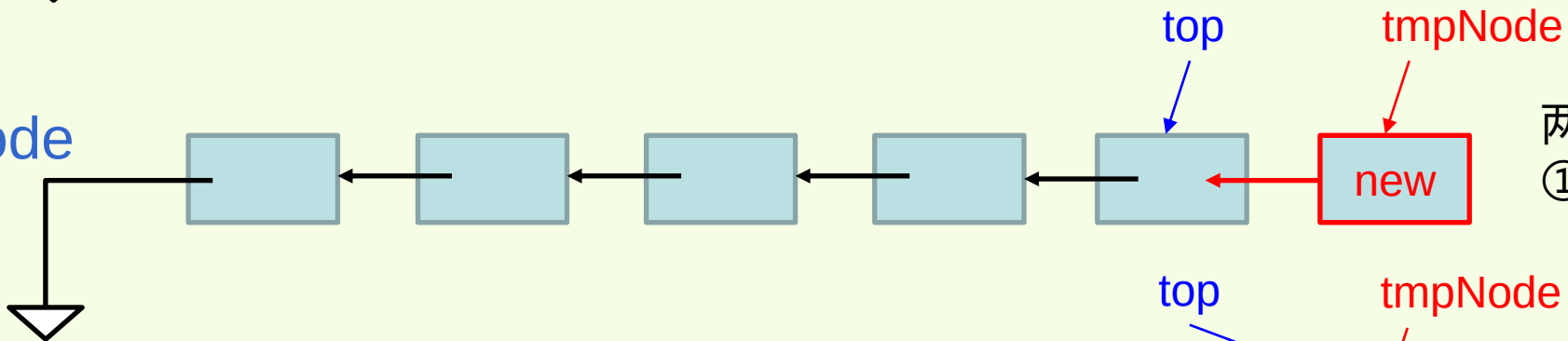
- 方法3：设置一个计数器。



Linked Stack

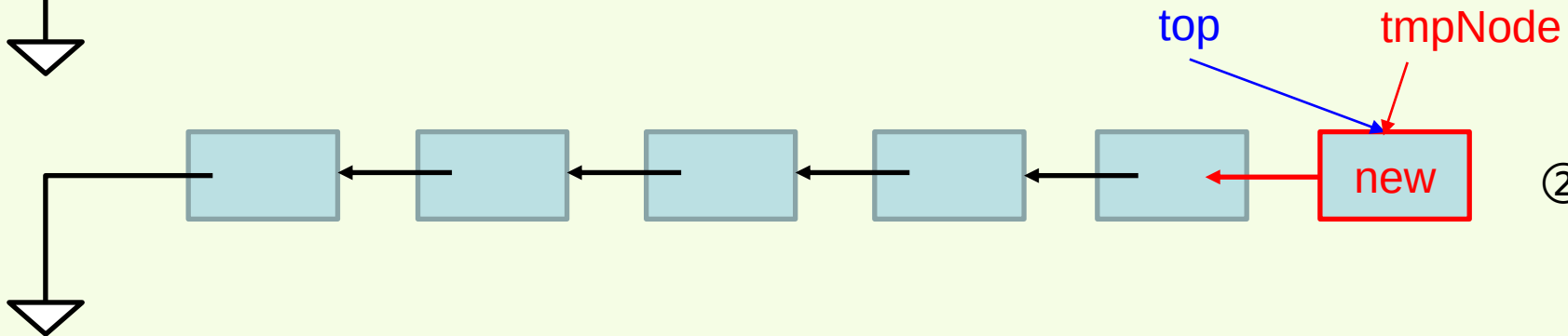


- Push a node



两步:

① 新增结点, 它的next指针指向原栈顶结点

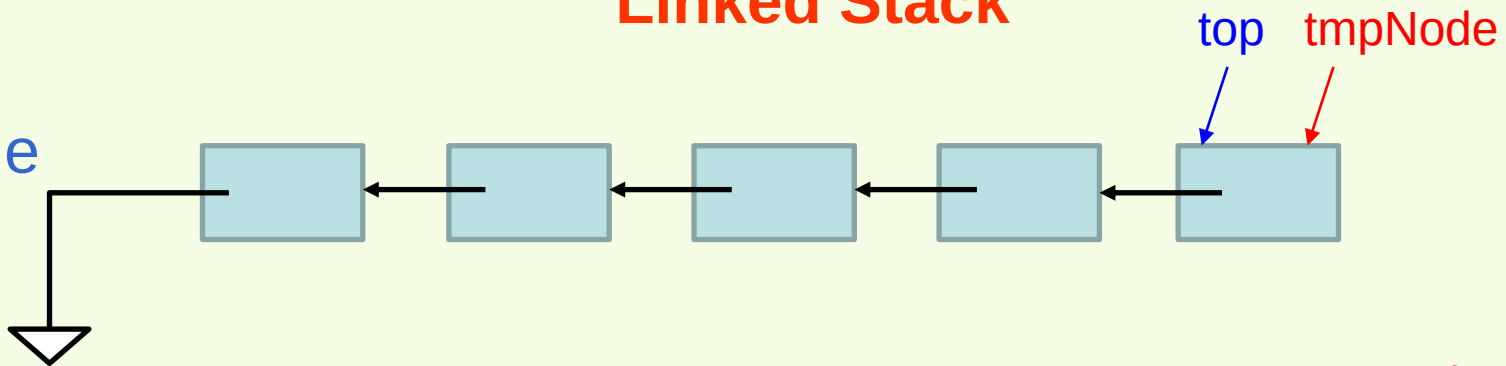


② 调整top指针



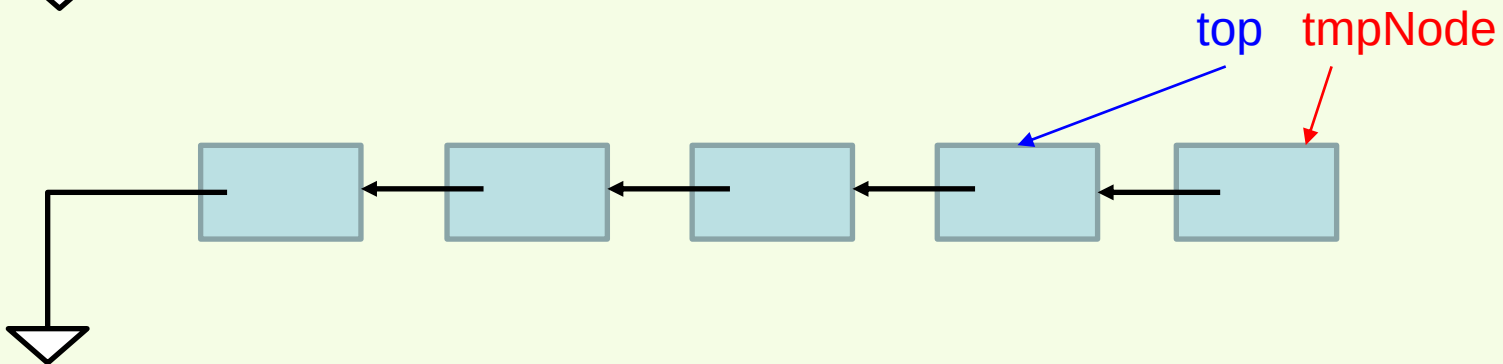
Linked Stack

- Pop a node

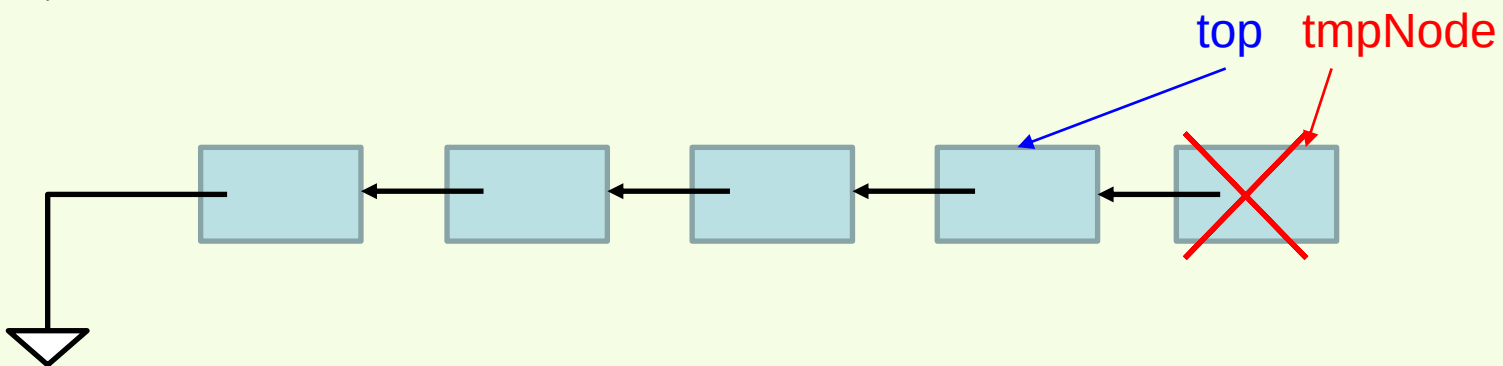


三步:

① 临时指针指向栈顶结点&读取该结点数据



② 调整top指针

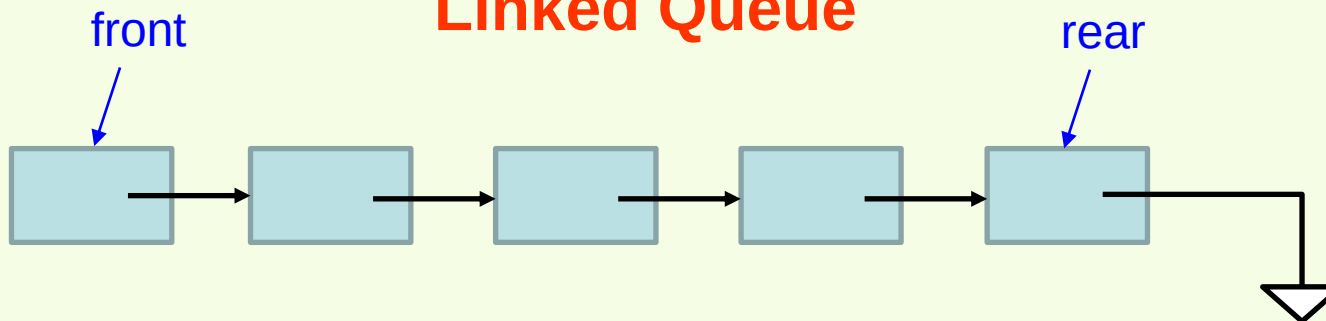


③ 释放临时指针指向的结点

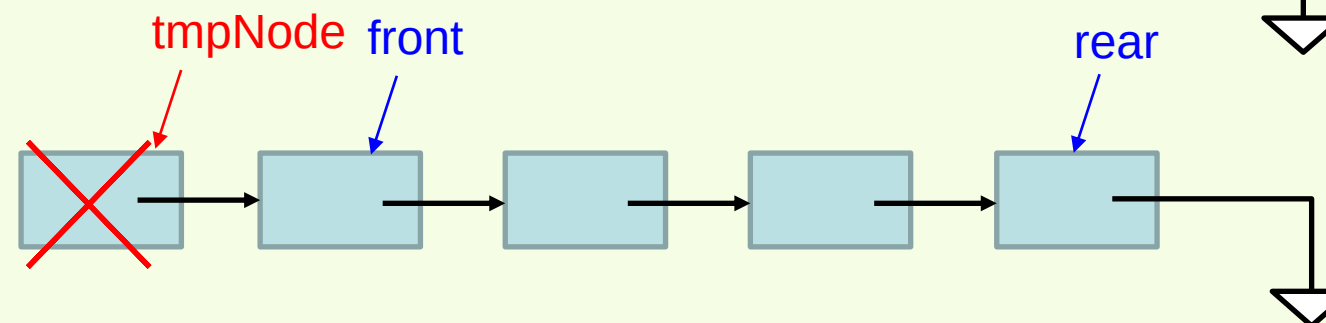
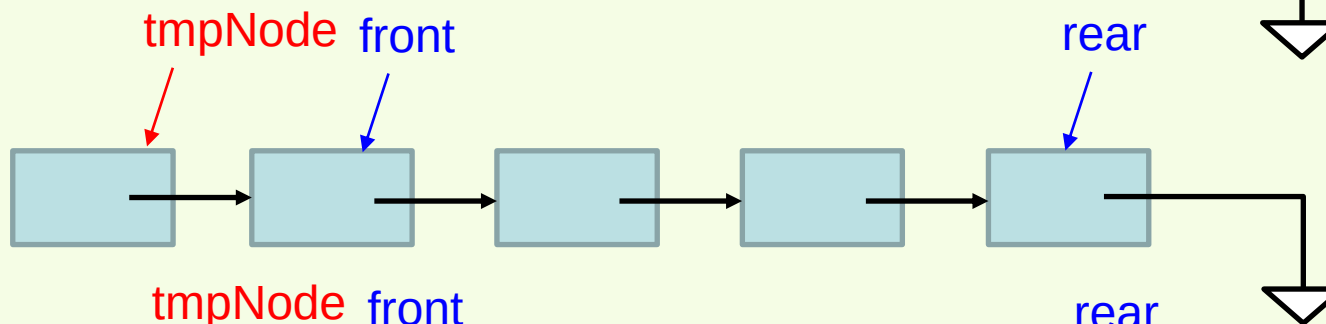
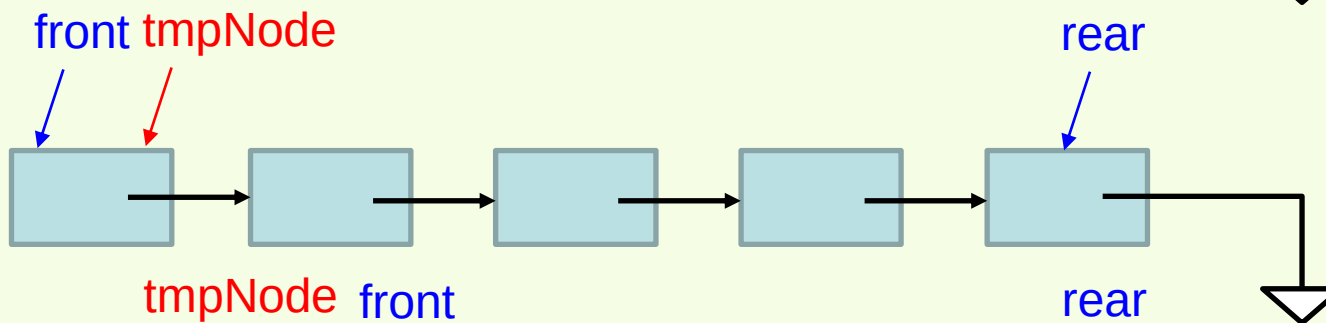


Linked Queue

A linked queue



• Dequeue a node



三步:

① 临时指针指向队头结点&读取该结点数据

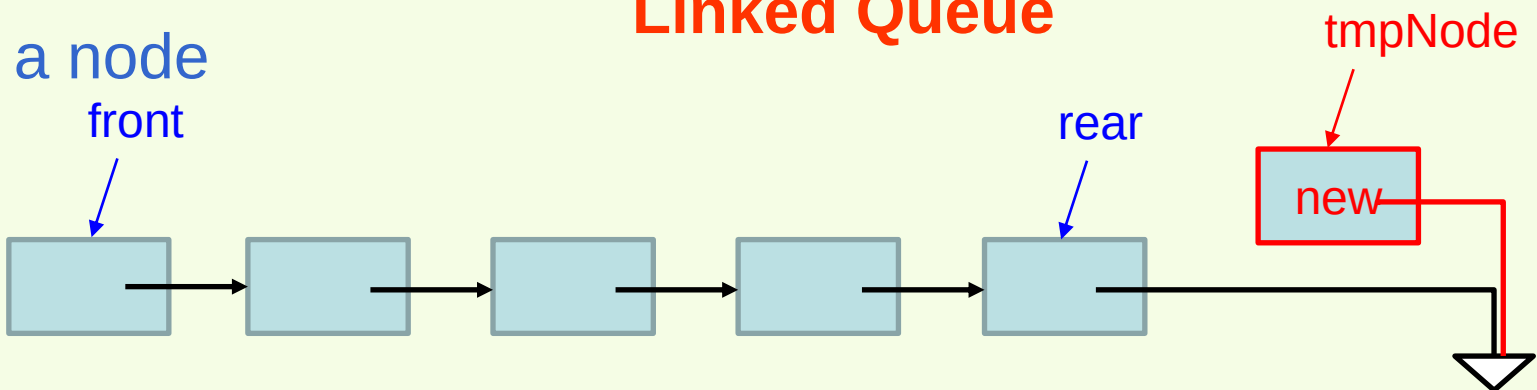
② 调整front指针

③ 释放临时指针指向的结点



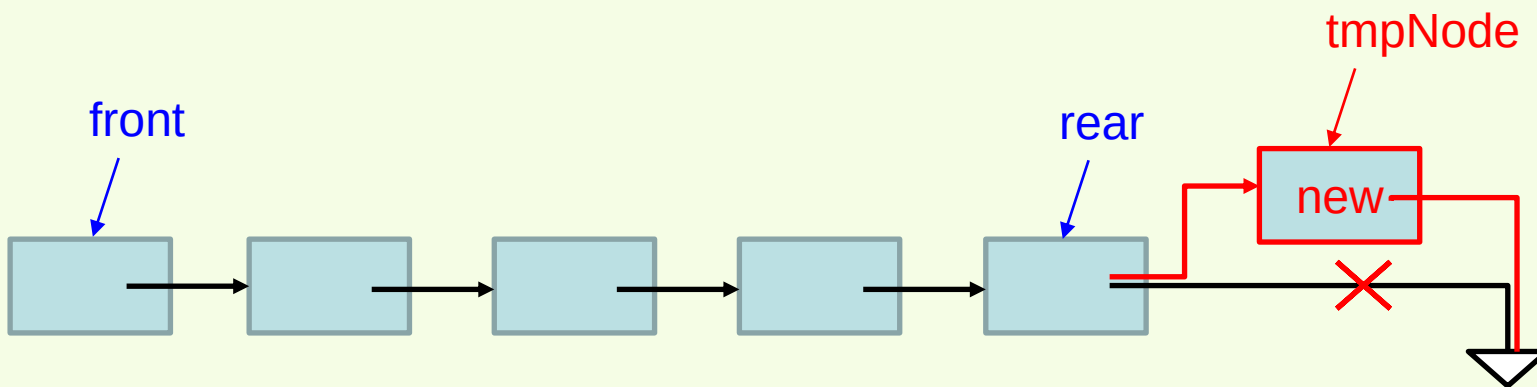
- Enqueue a node

Linked Queue

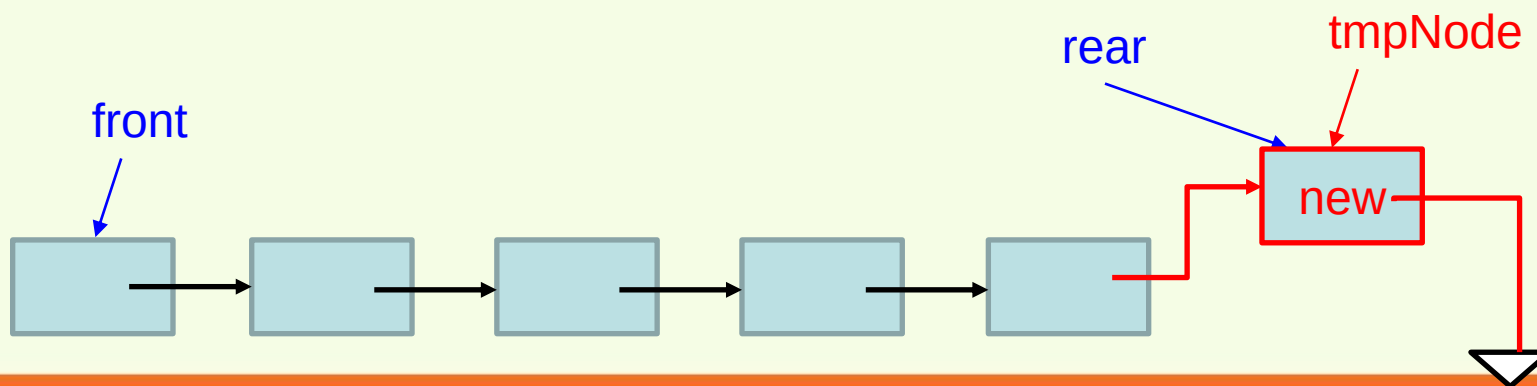


三步:

① 新增结点, 它的next指针指向NULL



② 修改原队尾结点的next指针指向新结点



③ 调整rear指针